Kubernetes as a Virtual Machine Management System

A Comparative Study of KubeVirt and Traditional Approaches

EMIL KARLSSON

Master in Computer Science
Date: February 29, 2024
Supervisor: Han Fu
Examiner: Cyrille Artho
School of Electrical Engineering and Computer Science
Host company: Net Insght AB
Swedish title: Swedish title TBD

Abstract

Sammanfattning

Contents

1 Introduction **2**
1.1 Background 2
1.2 Research Question 2
1.3 Problem 3
1.4 Purpose 3
1.5 Goals 3
1.6 Research Method 3
1.7 Limitations 3
1.8 Structure of the thesis 3

2 Background **4**
2.1 Virtualization Fundamentals 4
2.2 Understanding Hypervisors 6
2.3 VM Management Essentials 8
2.3.1 Comparison of VM Management Systems 9
2.4 Containers and Kubernetes: A Paradigm Shift 13
2.4.1 Containers Explained 13
2.4.2 Orchestration Using Kubernetes 14
2.5 Containers vs. Virtual Machines 16
2.5.1 Comparative Overview 16
2.5.2 Decision Factors 18
2.5.3 Towards Stronger Container Isolation 20
2.6 Extending Kubernetes with KubeVirt 21
2.6.1 KubeVirt and GitOps 22
2.6.2 Comparison with Traditional Approaches 22
2.6.3 Features and Capabilities 23
2.7 Evaluating VM Management Systems 23
2.7.1 Strategies for Evaluation 23
2.7.2 Metrics for Comparison 23

2.8 Related Work 23

3 Methods **24**

4 Results **25**

5 Discussion **26**

6 Conclusions **27**

Bibliography **28**

A Something Extra **36**

;

Chapter 1

Introduction

1.1 Background

1.2 Research Question

1. What performance, scalability, and operational efficiencies can be gained by integrating virtual machines in Kubernetes?

   This could stem from using a single resource scheduler. Several metrics will be used to evaluate this, as described in the background and related work.

2. What are the current challenges and limitations in managing virtual machines and containerized applications in separate environments?

   e.g. running CloudStack/OpenNebula/OpenStack alongside Kubernetes.

3. How can system administration be simplified by running virtual machines in Kubernetes using KubeVirt?

   e.g. if there are any efficiencies to be gained in terms of usability, for instance:

   - After clicking *Deploy VM*, how quickly can the virtual machine be accessed?
   - How many steps are there before I can access my virtual machine?
   - Are there any perceived benefits when interacting with the system, compared to the previous one?

1.3 Problem

1.4 Purpose

1.5 Goals

1.6 Research Method

1.7 Limitations

1. This project will not investigate any solutions for shortcomings in KubeVirt itself, for example, finding alternatives for features that exist in existing virtual machine management systems but not in KubeVirt. As such, this project acknowledges that KubeVirt might not be a complete replacement for traditional virtual machine management tools (also described in [1]), as it might not offer all the features of an IaaS system. Therefore, only features that exist in KubeVirt and the compared system will be considered when testing. However, any lack of features (KubeVirt or not) will be brought up in the discussions.

2. From the description of this project above, there are also mainly two scenarios that will be evaluated.

   - A system uses traditional virtual management (OpenNebula, CloudStack etc.).

     In this scenario, Kubernetes with KubeVirt is seen mainly as a 1:1 replacement.

   - A system uses both traditional management and Kubernetes.

     This scenario is more aimed towards unifying the environments using KubeVirt.

1.8 Structure of the thesis

Chapter 2

Background

In this chapter, we will discuss how virtualization has evolved to reach the point where we are today.

2.1 Virtualization Fundamentals

Virtualization is a technology [2] that enables partitioning or grouping computing resources into operating environments using methods such as partitioning, emulation, and time-sharing. Virtualization could occur in various forms, such as network virtualization, storage virtualization, and server virtualization. More commonly used, though, is hypervisor-based virtualization [3], which is a software layer that creates virtual operating environments, more commonly referred to as virtual machines (VMs).

A VM is an emulation of a physical computer [4] and is perceived as *real* computer by the application running inside it even though underlying resources are, more often than not, shared with other VMs. For this reason, virtualization is often used to improve resource utilization and to isolate applications from each other. Such a setup improves the flexibility of the computing environment, but it implies that the underlying mechanism is formally different from that of a physical machine. For instance, hardware abstraction is used to decouple the VM's perceived hardware capabilities from the physical hardware. Abstracting the underlying hardware allows for *virtual* computing resources, such as virtual CPUs and virtual memory, either by grouping or partitioning the physical resources.

Furthermore, the abstraction layer can exist in multiple different layers depending on the virtualization technology used, ranging from the hardware level to the application level, some of which are listed below:

- Full virtualization (FV): Provides a complete emulation of the underlying physical hardware and creates a complete virtual environment. FV enables fully decoupling the VM from the underlying resource and does not require any modification [2][5][6].

- Paravirtualization (PV): A subset of FV aiming to improve the VM performance by reducing the number of resources that are emulated. The VM is presented with an image that is similar, but not identical, to the underlying hardware. As such, the guest OS is aware it is running inside a VM [5][6].

- Application virtualization (AV): A method where an application is encapsulated in a virtualized lightweight environment, such as it is perceived it is being run on a specific platform. Application virtualized using AV acts as a portable and self-sufficient unit, thus often used to package and distribute applications, such as the JVM [2][7].

Define virtualization and discuss its significance in modern IT infrastructures, including the evolution from physical to virtual resources.

Benefits and Challenges

Many of the benefits of virtualization stem from the fact that there exists an abstraction layer above the physical hardware. The abstraction layer decouples software from hardware, e.g. vendors with different I/O devices can be used in the same environment, allowing for a more flexible and scalable computing environment as a whole where hardware is turned into a simple pool of computing resources [4]. As such, there are numerous benefits of virtualization, a few of which are listed below:

- Resource utilization: Virtualization allows for better utilization of hardware resources, as multiple VMs can run on the same physical hardware. This is achieved by sharing the underlying resources, such as CPU and memory, between the VMs [2].

- Isolation: VMs can isolate their entire state from one another, including the operating system (OS), filesystem and application stack. Isolation is also beneficial in scenarios where reproducibility is key, such as in testing and development environments [8].

- Flexibility: VMs not tied to a specific hardware, and can be moved between different physical resources.

- Management: VMs can be created and destroyed on demand, and can be managed using software tools [2].
- Virtual hardware: A VM can be configured with hardware that does not exist in the physical environment, such as a virtual network card or a virtual hard drive [2].

However, the portability made possible by virtualization comes at a cost of performance. Since, at the core, a VM is a complete but emulated system, it means that every instruction issued must be interpreted in software. As such, despite what level of abstraction layer is used, there is an inescapable added overhead when using virtualization. A common hypervisor such as KVM (see 2.2) introduces an overhead of 5-10% [9], and the overhead can be even larger when using a full virtualization approach.

Outline the advantages virtualization offers, such as resource optimization and isolation, alongside potential challenges, such as complexity and overhead.

2.2 Understanding Hypervisors

A hypervisor is a software layer that enables managing multiple VMs on a single physical machine by allowing multiple operating systems (termed as *guests*) to share the same hardware [3][10]. Hypervisors, often referred to as virtual machine monitors (VMMs), are responsible for creating and managing VMs, and for providing the necessary abstractions to the VMs.

Hypervisors can be categorized into two types, *type 1* and *type 2*, based on their operational context and how they interact with the underlying hardware [10][11]. While the goals and purpose of type 1 and type 2 hypervisors are the same, there are some core differences. type 1 hypervisors, commonly referred to as *bare-metal* hypervisors, run directly on top of the physical hardware and do not require an OS underneath. A type 1 hypervisor is therefore scalable to terabytes of RAM and hundreds of CPU cores [11]. However, without any other additional software, it is also responsible to schedule and allocate required system resources to the VMs. type 2 hypervisors, also known as *hosted hypervisors*, rely on an underlying OS to manage the calls to the CPU and memory resources. The presence of an underlying OS inevitably introduces latency, which is why type 2 hypervisors are often used for development and testing purposes [10].

Furthermore, hypervisors can be further categorized into *monolithic* and *microkernelized* hypervisors [12]. Monolithic hypervisors are built as a single

large software stack, managing all the VMs and their resources, and hosting driving for all hardware access. Having all drivers reside within the hypervisor means that the hypervisor will only run on selected hardware. However, it also means no operating system is required at all, as the hypervisor acts as an operating system by supporting all the operations of its VMs. Microkernelized hypervisors, on the other hand, do not require device drivers to be present in the hypervisor. Instead, the drivers are installed in a form of managing VM, often referred to as parent VM. Each VM created by such a hypervisor is thus referred to as a child VM, as it uses the parent VM's drivers to access the hardware. This approach is more flexible, as it can allow the hypervisor to run on any hardware that supports the parent VM's operating system.

In the realm of hypervisors, the concept of *security rings* is often used to describe the level of privilege different part of a hypervisor has. A security ring represents a hierarchical protection domain, where the innermost ring (ring 0), usually reserved for the OS kernel, has the highest privilege, and the outermost ring (ring 3), usually reserved for user applications, has the lowest privilege. The hypervisor itself is often placed in ring 0, its VM kernel space in ring 1, and the VM user space in ring 3. Security rings are thus used to define execution level to provide a system to limit the execution capabilities of VMs to regulate access to shared memory. This enabled a strong isolation layer between the VMs, and between the VMs and the hypervisor, thus a key part of a hypervisor's security model [12][13].

Explain what hypervisors are, differentiating between type 1 (bare-metal) and type 2 (hosted) hypervisors, and their operational contexts.

Key Hypervisors

There are several hypervisors available, each with its own set of features and capabilities, some of which are listed below:

- VMware ESXi: A lightweight type 1 monolithic hypervisor often used in enterprise environments. ESXi is known for its ultra-lightweight footprint stretching merely a few hundred megabytes, and its ability to run directly on the hardware without an underlying OS [12].

- Microsoft Hyper-V: A type 1 microkernelized hypervisor that is part of the Windows Server operating system. As a microkernelized hypervisor, Hyper-V differentiates between parent and child VMs and uses the parent VM to manage hardware access for the child VMs [12][14].

- Citrix Hypervisor: Formerly known as XenServer, is a type 1 microkernelized hypervisor based on the Xen Project. It runs using a modified Linux kernel in a VM, called Dom0, that manages hardware I/O and interacts with the other virtual machines, called DomU guests [6][12].
- KVM: A type 1 hypervisor that is part of the Linux kernel. As such, KVM uses most of the functionalities from the Linux kernel, such as the scheduler and memory management. KVM has become a popular alternative to proprietary hypervisors, such as VMware ESXi and Microsoft Hyper-V, as it is open-source and free to use [12][15].

Delve into specifics of widely used hypervisors such as VMware's ESXi, Microsoft Hyper-V, and KVM, highlighting their features, performance aspects, and use cases.

2.3 VM Management Essentials

A management system for VMs involves a set of tools and processes that are used to create, deploy, and manage VMs [16]. The management system is responsible for managing the entire life cycle of a VM, including provisioning, deployment, monitoring, and scaling. Not be confused with a hypervisor, a VM management system lives on a higher level and aims to provide an abstraction layer in the form of an interface to allow interaction with the VMs, often enabling the support of a set of hypervisors [17]. As such, a VM management system unifies hypervisors, typically through a higher-level API, such as a CLI or a web-based user interface.

Concepts and Features

With a higher abstraction layer, a VM management system brings a set of concepts to simplify managing VMs, such as clustering, availability, and scalability [17]. Clustering includes grouping a set of resources, such as VMs or storage nodes, to provide an image that is perceived as a single resource. Availability refers to the ability to keep a resource active or running, which is, more often than not, clustered resources. Scalability involves technologies to ensure resources can manage an increase or decrease in computational load, such as increasing the amount of virtual cores in a VM if it's overloaded.

Infrastructure as a Service

A VM management system is often part of a larger cloud computing infrastructure, such as Infrastructure as a Service (IaaS). IaaS is a lower-level cloud computing model that aims to eliminate the need to install and manage physical hardware by providing virtualized computing resources, such as VMs, virtual networks and storage.

Discuss essential VM management capabilities, including live migration, snapshot management, and scalability.

2.3.1 Comparison of VM Management Systems

Managing VMs can be done in many ways, and there are many VM management systems available, each with its own set of features and capabilities. Many such systems are a part of a larger platform, such as an IaaS or even as large as a cloud platform. Some of the most prevalent platforms that have been used in the industry are OpenStack, CloudStack, and OpenNebula, and some of the most popular public cloud platforms are Amazon EC2, Azure Virtual Machines, and Google Compute Engine.

OpenStack

An open-source IaaS platform that offers a large set of features to manage VMs, network, and storage [16]. OpenStack's architecture is inherently modular, and it is designed to be deployed in a distributed manner [18]. The architecture includes three major parts:

- *Nova*: The compute service that manages VMs and provides an interface to manage VMs. Nova supports all operations regarding the life cycle of a VM, such as creating, deleting, and migrating VMs. Nova is itself divided into multiple components, each employing a *shared-nothing* and *messaging-based* policy, which means every component is independent and communicates with other components using a message queue.

- *Swift*: The object storage service that provides a scalable and redundant storage system for VMs. It is a long-term storage for static data, such as images and backups, and is comparable to Amazon S3.

- *Glance*: The image service that provides a catalog of VM images that can be used to create new VMs.

OpenStack supports features such as live migration, snapshot management, and scalability, and is often used in large-scale cloud environments [16]. However, these features are often hypervisor-specific and require both support from the hypervisor and specific configuration to work [19].

CloudStack

An open-source IaaS platform and cloud computing software that manages computing, networking and storage resources, and is used to build and manage public and private cloud environments [20]. On a high level, CloudStack is built in a monolithic fashion, encapsulating all the management features into two software packages called the management server and agent. The management server is responsible for managing the life cycle of VMs, including provisioning, deployment, and scaling, and is comparable to a control node. The agent, however, is installed on each node designated to running VMs and is responsible for low-level management tasks of the VMs on that node, such as interacting with the underlying hypervisor. While it is a monolithic system, CloudStack is designed to be horizontally scalable, and can be deployed in a distributed manner with multiple management servers [20].

CloudStack supports a large variety of hypervisors, such as VMware, KVM, and XenServer, and storage services, such as Amazon S3, allowing it to function as an abridgment layer for computing, network, and storage to unify a wide range of existing infrastructure [20]. Features such as live migration, snapshot management, and scalability are supported but are often hypervisor-specific and require specific configuration to work [21].

OpenNebula

An open-source IaaS platform and toolkit used to provide open, scalable, and comprehensive management of virtualized data centers [16]. OpenStack employs a modular architecture divided in three layers [22][23]:

- *Tools layer*: Provides a set of interfaces such as a CLI and web-based user interface to interact with the system, but also high-level management tools such as scheduling.
- *Core layer*: Provides the core functionalities of the system, such as host management, VM management, and virtual network management.
- *Driver layer*: Provides an abstraction on top of the underlying infrastructure to enable the system to support a wide range of hardware, including networking, storage, and hypervisors. As such, the driver layer

is responsible for allocation, deallocation and monitoring of resources, such as VMs and storage.

By dividing the system into these layers, OpenNebula provides modularity that enables the system to be extended and customized to fit specific needs, such as integrating with other systems or using OpenNebula to build a private cloud.

Amazon EC2

Amazon Elastic Compute Cloud (EC2) is a web service that provides on-demand compute resources over the cloud [24][25]. EC2 is part of Amazon Web Services (AWS), is designed to be scalable and flexible, and is often used to host general-purpose applications, including web applications, databases and enterprise applications. While the EC2 platform uses Xen-based hypervisors (see 2.2) under the hood [26], it acts as a complete abstraction layer over the underlying hardware and hypervisors and instead provides a rich set of tools to manage the VMs.

EC2 provides a large set of pre-configured templates called Amazon Machine Images (AMIs) that can be used to create new VMs, as well as a configuration in terms of specifications. This enables EC2 to provide a wide range of VMs, such as general-purpose, compute-optimized, memory-optimized, and storage-optimized VMs [24]. Features such as live migration, snapshot management, and auto-scaling are supported,

Azure Virtual Machines

Azure Virtual Machines is a service provided by Azure to manage VMs in the cloud [27], and is designed to be a cost-effective alternative to on-premises hardware [28]. Azure Virtual Machine provides a set of preconfigured VMs that can be used to create VMs for specific purposes, such as general-purpose or high-performance computing (HPC).

Behind the scenes Azure Virtual Machines runs on Hyper-V (see 2.2) and provides a set of features such as live migration, snapshot management, and auto-scaling [29].

Google Compute Engine

Google Compute Engine is a service provided by Google Cloud to manage computing resources in the cloud [30]. Compute Engine consists of three major parts: VMs, storage, and networking, where VMs are the core component.

VMs are offered in a wide range of configurations, such as standard, high-memory, and high-CPU, making them suitable for different computational workloads [31].

Compute Engine uses KVM as its hypervisor (see 2.2), and provides many features such as live migration, snapshot management, and auto-scaling [30][26].

Comparison

OpenStack, OpenNebula, and CloudStack are all examples of software that can be used to build both private and public cloud platforms [32][33]. A private cloud is an on-premise cloud that is hosted within an organization and is, more often than not, dedicated to internal use. A private cloud, on the other hand, is a cloud platform meant to facilitate external users. A common denominator for these systems is that they are open-source and that they are highly customizable and extendable. As such, much of the configuration and management of the system is done by the user, and the system can be integrated with other systems to fit specific needs. This means that many of the conveniences such as preconfigured VMs and features such as live migration often require specific configuration and setup.

Amazon EC2, Azure Virtual Machines and Google Compute Engine are examples of VM management systems in public cloud platforms. They are designed to be more user-friendly and to provide a more complete abstraction layer over the underlying hardware and hypervisors [32]. In comparison, these technologies can be seen as complete setups of the open-source platforms previously mentioned, with all the configuration and management done by a provider. As such, many of the conveniences and features are provided out of the box and are perceived as hypervisor- and hardware-agnostic. This means that the user does not need to worry about the underlying hardware and hypervisors, and can instead focus on managing the VMs. This creates a trade-off where the user has less control and customization but instead gains a more user-friendly and complete system.

Both private and public cloud solutions for VM management are designed to be scalable and flexible and support a wide range of use cases (although private cloud solutions scale worse in practice [33]). Which solution to use depends on the specific needs of the organization, and the level of control, customization needs, and the required security level of the system.

Briefly introduce traditional VM management systems like OpenStack, CloudStack, and OpenNebula, focusing on how they implement these core features.

2.4 Containers and Kubernetes: A Paradigm Shift

Besides the common cloud service models (IaaS, PaaS, and SaaS), another service model has emerged, called *Container as a Service (CaaS)* [34]. CaaS is a container-based cloud service model that has gained significant traction due to the lightweight and portable nature of a container, allowing for simple application bundling that can run in a vast range of environments.

Containers are often used in conjunction with a container orchestration platform, such as Kubernetes, to manage the life cycle of the containers including deployment, scaling, and monitoring. As such, they are often used to build, test, and deploy applications consistently. A common use case for containers is to run microservices, which is a software architecture that employs a set of small, independent and loosely coupled applications [35].

2.4.1 Containers Explained

Containers are a form of virtualization that offers a horizontally scalable and lightweight alternative to VMs [36][37]. Unlike VMs, containers do not require a hypervisor to run, and instead run on top of an OS and share the OS kernel with other containers. By not requiring a hypervisor, containers can run with less overhead than VMs, making their life cycle relatively fast and efficient compared to VMs.

A container is created from a container image, which is a lightweight executable software package that can run a specific application including dependencies and data files. By encapsulating an application and its dependencies in a single package, the container image becomes portable and independent, allowing for the container to run consistently across different environments. A container image is created by describing its parent image, such as an OS image, and the steps to install and configure the application and its dependencies in a file, such as a Dockerfile. A parent image can itself have a parent image, creating a chain of dependencies referred to as layers. Managing the container image as a series of layers allows structured and transparent dependency management and efficient caching [38][39][40].

Container images are often stored in a container registry. A container registry provides central storage for container images and is used to distribute container images to environments that can run containers, such as a Kubernetes cluster. Common features for a container registry include versioning, access control, image scanning, and features to integrate with software devel-

opment cycles, such as triggers to notify an external system when a new image is pushed to the registry [41][42].

The Open Container Initiative (OCI) is a standardization effort that was created to ensure that containers are portable and interoperable across different platforms [43]. The OCI standard defines a runtime specification, image specification, and distribution specification, allowing for a container to be consistent in terms of its runtime environment, image format, and distribution method. Moreover, a container runtime, such as containerd, is used to create, deploy, and manage containers on a node [44]. Most runtimes, including containerd, implement the OCI runtime specification and can consistently run containers. This suggests that the OCI can be viewed as an abstraction layer, where the OCI runtime specification acts as an intermediary between the container runtime and the OS kernel and the container image, and the distribution specification acts as an intermediary between the container runtime and the container image registry.

Define containers, their architecture, and how they differ from VMs in terms of overhead and isolation.

2.4.2 Orchestration Using Kubernetes

Kubernetes is an open-source container orchestration platform that manages containers. It maintains a set of abstraction layers to manage the life cycle of containers and to provide a consistent environment for the containers to run in, such as scheduling, scaling, and networking [37].

Kubernetes is inherently declarative, meaning that instead of issuing commands to perform a specific task, such as starting a container, Kubernetes is given a desired state for a set of resources, and it is responsible for ensuring that the current state of the resources matches a desired state [37][45]. For this reason, Kubernetes provides auto-restart, re-schedule, and replication control to ensure a resource stays in the desired state. By specifying the desired state, an entire application can be contained in a single configuration file, including its container, networking, storage, and security requirements.

Some of the abstractions used in Kubernetes include *Pods*, *Services*, and *Deployments* [45]. A Pod is the smallest deployable unit in Kubernetes and represents a group of one or more containers that are scheduled to run on a node. A Service represents a single-point-of-entry communication layer for a set of Pods and is used to provide a consistent way to access the Pods, such as load balancing. A Deployment is a higher-level abstraction on top of Pods that is used to manage the life cycle of the Pods, such as scaling and rolling

updates. A Deployment does not represent an executable resource but instead describes a desired state in which a set of Pods should be. There are many other abstractions in Kubernetes, each with its own set of features and capabilities that together create a feature-complete platform for container orchestration.

A Custom Resource Definition (CRD) is a form of extension for Kubernetes resources that allows for the creation of custom resources [45][46]. A CRD is used to extend the Kubernetes existing abstractions to provide additional features and capabilities to allow for a more feature-complete platform when custom behavior is needed. For instance, a CRD can be used to enforce certain fields to be specified to create the resource by applying custom validation rules [47].

Kubernetes uses a client-server architecture, where the server, often referred to as the control plane, is responsible for managing the state of the cluster, and the client, often referred to as the worker node, is responsible for running containers [45]. The control plane runs system-related services, such as the API server, scheduler, and controller manager, and is thus responsible for managing the state of the cluster. The API server can be viewed as the primary service for a Kubernetes cluster as it is the main management point for the cluster, receiving and processing requests from the client. The worker node, on the other hand, is responsible for running the containers. The worker node consists of two primary components: the kubelet and the kube-proxy. The kubelet is a service that includes a container runtime, which runs containers according to requests by the control plane. Meanwhile, the kube-proxy service manages the network communication for the containers. To interact with Kubernetes from the perspective of a user, a client is used to send requests to the API server. The client can be a CLI, such as kubectl, a web-based user interface, or a programmatic interface such as a REST API. The CLI tool kubectl is the most common way to interact with Kubernetes and is used to create, delete, and manage resources in the cluster [45].

Through its abstractions, life cycle management, and declarative client-server architecture, Kubernetes provides a consistent and feature-rich platform for container orchestration.

Introduce Kubernetes, detailing its components, functionalities, and how it revolutionized container management.

2.5 Containers vs. Virtual Machines

Virtualization has enabled the commoditization of computing resources due to the benefits of being able to map virtual hardware to physical hardware (see 2.1) and has been key in enabling cloud computing [37][48]. It has enabled the creation of flexible and scalable computing environments and has been used to build and manage large-scale cloud platforms. Virtualization has mainly brought two technologies, hypervisors and containers. Hypervisors are used to create VMs, including all associated aspects such as an OS. Containers, on the other hand, are used to create a lightweight and portable environment for an application to run in and are often run inside VMs. As such, a container comes with less overhead when compared to a hypervisor. However, by its nature, a VM can provide a more isolated environment than a container, leading to a more secure environment. Thus, the benefits of containers do not come without a cost, and there is a trade-off between the two technologies; one is lightweight, portable, and quick to start, while the other is more independent and secure.

2.5.1 Comparative Overview

While containers and hypervisors are both technologies brought by virtualization, they operate on distinct levels of abstraction [48]. Containers operate on the application level using OS-level virtualization, which leads to sharing parts of the OS kernel with other containers. A hypervisor on the other hand can virtualize the entire hardware stack, including the entire OS, depending on the type of hypervisor used (see 2.2). As such, the two technologies have different use cases and are often used in conjunction with each other.

The main difference between containers and VMs is the level of isolation they provide. Containers share the OS kernel with other containers while VMs run on their OS. While this means a container can share resources more efficiently, it also means that a container is subject to a larger attack surface, as a vulnerability in the OS kernel can affect all containers running on the same host. VMs, on the other hand, reduce the attack surface by running more independently, meaning a compromised VM is less likely to affect other VMs running on the same host [49].

Another aspect of isolation is how co-located containers and VMs can share resources. When sharing resources, such as CPU and memory, containers are vulnerable to the noisy neighbor problem. The noisy neighbor problem occurs when a resource-sharing mechanism, such as a scheduler, is

unable to provide a fair share of resources. Some scenarios include when a container can consume all available resources due to a heavily loaded application, or when a container intentionally consumes all available resources to starve other containers. Solutions exist to mitigate the noisy neighbor problem, such as limiting the number of resources a container can consume through OS-level features such as *cgroups*, a feature in Linux designed to limit, prioritize, and isolate the resource usage of a set of processes [50][51][52]. VMs, on the other hand, can provide a fair share of resources, as they provide a more isolated environment, leading to more resilient and predictable performance. However, the isolation does not mitigate the noisy neighbor problem for all resource types. For instance, a VM is still vulnerable to the noisy neighbor problem when sharing network-related resources, such as network bandwidth and network-based storage [48].

Overcommitment is a technique used to improve packing efficiency, often referred to as *consolidation ratio*. Containers use the overcommitment mechanisms built into the OS itself, such as CPU overcommitment by multiplexing and memory overcommitment through virtual memory and memory swapping. Hypervisors, on the other hand, map virtual hardware devices to physical hardware, creating virtual computing resources that are not directly tied to the physical resources. This works well for the CPU by creating virtual CPU (vCPU) cores. However, memory overcommitment is a more complex issue in hypervisors than in containers due to the way hypervisors allocate memory. Unlike containers, hypervisors assign a static amount of memory to a VM when it is created, and only reclaim the memory when the VM is deleted. Methods exist such as memory ballooning address the limitation [53], which is a technique used to treat memory as a shared resource across VMs and allow the hypervisor to reclaim memory from VMs when it is needed elsewhere, thereby reducing the need for memory overcommitment [48][54].

Live migration is a technique used to move a running resource from one host to another without interrupting the services provided by the resource. The technique is often used to balance the load or to perform maintenance on the hosting nodes. For VMs, it is a mature and widely technique for large data centers. Live migration works by copying memory pages from the source host to the destination host and then transferring the control of the VM from the source hypervisor to the destination hypervisor. Since VM disks are often stored on a shared storage, such as a Storage Attached Network (SAN), the VM disk is not transferred but rather reattached after the live migration is finished. For this reason, the VM can safely continue to run after its running state has been transferred to the destination host [55]. Live migration for containers,

on the other hand, is not as mature as for VMs. Unlike VMs, containers require OS-level process migration, which is less reliable. However, several projects aim to provide live migration for containers, such as Checkpoint-restore in Userspace (CRUI). CRUI operates by checkpointing the state of a container, such as its memory and file descriptors, and then restoring the state on another host. While this is considered as container live migration, it is more comparable to snapshot and restore operations than a live migration operation, as CRUI freezes the container during the checkpointing, and converts the state of a container to a collection of files on disk, that can be used to restore the container on another host [56][57][58]. Another approach for live migrating containers involves terminating them on one host and restarting them on another. This strategy assumes the container is stateless or utilizes shared storage for its state, as is often used in Kubernetes since it does not provide container live migration. Instead, Kubernetes facilitates persistent storage through Persistent Volume Claims (PVCs), essentially allowing a user to map portions of the container's state to shared storage, and assumes the rest can be discarded when a container is terminated [45].

Elaborate on the distinct characteristics of containers and VMs, including performance, isolation, and use cases.

2.5.2 Decision Factors

The choice between containers and VMs depends on the specific use case, and there are several factors to consider when deciding which technology to use [48]. Some of the core factors are security, scalability, and legacy support.

Security is a key factor when deciding between containers and VMs. Using containers means sharing the OS kernel, which implies that a vulnerability in the OS kernel could affect all containers running on the same host. Containers are thus, from an attack-surface perspective, more prone to some vulnerabilities than VMs, making VMs more suitable in scenarios that require a higher level of isolation. For instance when an application manages sensitive or confidential data [59]. Moreover, sandboxing is a security mechanism that is used to restrict an execution environment such that it can safely run untrusted code without affecting the rest of the system. For instance, when analyzing the effect of malware a sandbox environment ensures that the malware is not able to escape the restricted environment. Sandboxing is done in multiple layers, including limiting the resources a process can access, such as memory and CPU, and limiting the system calls a process can make. As such, a sandbox environment requires a high level of isolation, which makes VMs a natural fit

[60][61].

Configuration drift is another aspect to consider regarding security. It is a phenomenon that arises when the configuration of a system changes over time due to several factors including manual intervention and software updates, and can lead to an undefined state in a system that could create security holes [62][63]. Containers, as described in 2.5.1, are often run in a stateless manner using a single configuration file to describe the container, such as a Dockerfile. By being able to discard the state and recreate a container from a configuration file, the risk of configuration drift is reduced. As such, containers can provide a more consistent environment than VMs. Moreover, containers represent a more standardized and portable environment, using layers of dependencies, see Fig. By standardizing the environment, external tools are more easily able to provide security features such as vulnerability scanning [64].

When deciding between containers and VMs scalability is an important factor. One of the main benefits of containers is their ability to share the underlying OS. However, as described in 2.5.1, this also means that containers are more prone to the noisy neighbor problem, where co-located applications can cause performance interference. While this issue is related to isolation and security, it is more prominent in larger-scale environments, especially in multi-tenant environments [52][64].

Scalability is also related to cloud-native applications, which are applications that are designed to run in the cloud [65]. A cloud-native application is characterized by being horizontally scalable, stateless, and resilient, and if often built using microservices, see 2.4. Cloud-native applications thus require small and lightweight environments, making containers with an orchestration platform such as Kubernetes a natural fit.

Legacy support is another factor to consider when deciding between containers and VMs. Both VMs and containers can provide environments for legacy applications, such as applications that require a specific OS version or a specific set of dependencies. However, certain legacy applications are not able to run in a container due to the way they are designed, such as applications that require a specific OS kernel or a specific set of hardware. In such cases, a VM is often a better fit, as it enables virtualized hardware and a complete OS to be run [64][66].

Discuss factors influencing whether to deploy applications in containers or VMs, considering scalability, security, and application architecture.

2.5.3 Towards Stronger Container Isolation

Containers are often used in conjunction with VMs due to the lack of strong isolation properties out of the box, see 2.5.1. However, several projects aim to provide the isolation properties of a VM and the lightweight properties of a container, such as Firecracker, Kata Containers, and gVisor.

Firecracker is a KVM-based hypervisor that creates lightweight VMs, called microVMs. Compared to a traditional VM, a microVM can start in a fraction of a second using only a fraction of the memory, comparable to that of containers. This is achieved through heavy optimization both in the hypervisor and in the guest functionality, such as reduced overhead through lightweight I/O and a stripped-down guest OS [67][68].

Kata Containers is a project that, similar to Firecracker, aims to provide lightweight VMs that perform like containers while also providing strong isolation properties. Unlike Firecracker, Kata Containers uses both VMs and containers and replaces the container runtime, such as containerd (see 2.4.1) with the Kata runtime. The Kata runtime implements the OCI runtime specification to be interoperable with container ecosystems. However, instead of running the container directly on the host, the Kata runtime creates a VM, in which the container runs. This provides the isolation properties of a VM and the lightweight properties of a container. Kata Containers can achieve this by heavily optimizing the VM and its creation process towards running containers, as this is the only purpose of the VM [67].

Another approach to strong container isolation is gVisor, a user-space kernel that provides a secure environment for the container. Unlike both Firecracker and Kata Containers, gVisor does not use VMs to enhance the isolation properties but instead operates as an extension around the container. gVisor consists of two core components, *Sentry* and *Gofer*. Sentry is a user-space kernel that intercepts and handles system calls made by the container and can provide a secure environment for the container. Gofer is a file access component that provides the container with file access. Separating the file access from the syscall access allows for a more secure environment. For instance, a compromised Sentry does not imply read or write access to any file [69].

All three technologies aim to unify the main benefits of containers and VMs by providing environments that are both lightweight and secure. However, due to the varying added overhead, they are yet to be widely adopted [67].

2.6 Extending Kubernetes with KubeVirt

KubeVirt is a novel project that aims to provide a unified platform for managing both containers and VMs using Kubernetes. It is designed to naturally blend in with the Kubernetes ecosystem and provide a consistent interface to manage the life cycle of VMs, such as deployment, scaling, and monitoring. KubeVirt being built on top of Kubernetes means that it does not implement all the revolving components traditionally found in a VM management system, such as the scheduler, but instead uses the mature and feature-rich services the Kubernetes ecosystem provides [70]. By using CRDs (see 2.4), KubeVirt can extend the Kubernetes API to provide a specification for VMs to support features commonly found in traditional VM management systems, such as live migration, scaling and snapshot management. Moreover, CRDs allow KubeVirt to enforce validation on the resources to prevent invalid configurations, such as a VM that references a non-existing storage disk [71][72].

The design pattern chosen for KubeVirt's VM management is heavily inspired by other Kubernetes resources. A VM in KubeVirt is split into two resources called *VirtualMachine* and *VirtualMachineInstance*. The VirtualMachine resource is comparable to a StatefulSet and is used to describe the desired state of a VM, such as the VM's running state, CPU and RAM requirements and cloud-init configuration. The VirtualMachineInstance, on the other hand, is comparable to a Pod and is created by the VirtualMachine resource to represent the running state of the VM. As such, a non-existing VirtualMachineInstance represents a non-running VM, and an existing VirtualMachineInstance represents a running VM. The VirtualMachineInstance contains runtime information about the VM such as the VM's IP address, running state and resource usage.

Moreover, live migration and snapshots are implemented as CRDs, allowing for a declarative interface to manage these features. This is in stark contrast to traditional VM management systems, where live migration and snapshot management are implemented as imperative operations. For instance, by creating a LiveMigration resource, KubeVirt initiates a live migration operation [73][74]. This is comparable to the binding operation process for Persistent Volumes and Persistent Volume Claims, where the binding takes place when a PVC is created [75]

To enable the VMs to run as any other Kubernetes resource, KubeVirt implements a custom controller that is responsible for managing the life cycle of the VMs. A controller is a control loop that watches the state of certain resources and is responsible for ensuring that the current state of the resources

matches a desired state [76]. As such, the KubeVirt controller watches the state of the VirtualMachine resources and is responsible for creating and deleting VirtualMachineInstance resources to ensure that the current state of the VMs matches the desired state. For instance, when a VirtualMachine's CPU and RAM requirements are updated, the KubeVirt controller is responsible for updating the running VM to correctly reflect the new requirements.

Being built on top of Kubernetes, KubeVirt-specific resources can be managed using the kubectl CLI tool. This allows for the declarative creation, modification and deletion of VMs using the same interface as for other Kubernetes resources. However, certain operations are not possible to perform using kubectl. For instance, accessing serial or graphical consoles of a VM is not possible using kubectl. To enable these operations, KubeVirt provides a complementary CLI tool called *virtctl*. While virtctl primarily enables console access, it also provides convenience features such as live migration and snapshot management [77].

2.6.1 KubeVirt and GitOps

GitOps is a set of practices that enables using Git as a single source of truth for the desired state of the system, and it thus is heavily tied to the declarative nature of Kubernetes, as the desired state of the system is described in configuration files. By managing the infrastructure in Git, resources can be version controlled, and any changes to the system can be tracked and audited. This creates a developer-centric workflow, where developers can manage the infrastructure using the same tools and processes they use for managing application code [78]. Moreover, since GitOps is used for systems to manage their resources declaratively, it is a natural fit for KubeVirt. As such, KubeVirt opens up the possibility of managing VMs using GitOps, meaning that VM specification and life cycle management can be version-controlled and audited [**kubevirt-gitops**].

2.6.2 Comparison with Traditional Approaches

KubeVirt does not represent a new virtualization or hypervisor technology but instead leverages QEMU and KVM to run the VMs, which means KubeVirt is fundamentally similar to traditional VM management systems, such as OpenStack, CloudStack, and OpenNebula [79]. Therefore, KubeVirt can provide the same level of isolation and security as traditional VM management systems, as it uses the same underlying hypervisor technology. However, Kube-

Virt is not as feature-rich, as it is not designed to be a complete replacement for traditional VM management systems. For instance, OpenStack and oVirt consist of services to manage VMs and their interconnections such as network management, host provisioning and data warehousing [80]. KubeVirt, on the other hand, is primarily designed to manage VMs and their life cycle, such that it can provide a unified platform for managing both containers and VMs. As such, KubeVirt integrates well with Kubernetes to provide a seamless experience for VM management, including integrating with Container Network Interface (CNI) plugins to provide network connectivity for the VMs, and integrating with Persistent Volume Claims (PVCs) to provide storage for the VMs [71][81][82].

Provide an in-depth introduction to KubeVirt, its objectives, architecture, and how it integrates VM management within Kubernetes.

2.6.3 Features and Capabilities

Detail the features KubeVirt offers, such as VM life cycle management, network and storage integrations, and how it compares with traditional VM management approaches.

2.7 Evaluating VM Management Systems

2.7.1 Strategies for Evaluation

Outline methodologies for assessing VM management systems, including performance metrics, scalability tests, and operational efficiency.

2.7.2 Metrics for Comparison

Define specific metrics to be used in comparing KubeVirt against traditional VM management systems, such as deployment time, resource utilization, and ease of management.

2.8 Related Work

Chapter 3

Methods

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Chapter 4

Results

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Chapter 5

Discussion

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Chapter 6

Conclusions

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Bibliography

[1] Parth Yadav and Vipin Kumar Rathi. "KupenStack: Kubernetes based Cloud Native OpenStack". In: *arXiv preprint arXiv:2106.02956* (2021).

[2] Susanta Nanda Tzi-cker Chiueh and Stony Brook. "A survey on virtualization technologies". In: *Rpe Report* 142 (2005).

[3] Awodele Oludele et al. "On the evolution of virtualization and cloud computing: A review". In: *Journal of Computer Sciences and Applications* 2.3 (2014), pp. 40–43.

[4] M. Rosenblum and T. Garfinkel. "Virtual machine monitors: current technology and future trends". In: *Computer* 38.5 (2005), pp. 39–47. DOI: 10.1109/MC.2005.176.

[5] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Full and para-virtualization with Xen: a performance comparison". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.9 (2013), pp. 719–727.

[6] Tim Abels, Puneet Dhawan, and Balasubramanian Chandrasekaran. "An overview of xen virtualization". In: *Dell Power Solutions* 8 (2005), pp. 109–111.

[7] Fred Douglis and Orran Krieger. "Virtualization". In: *IEEE Internet Computing* 17.2 (2013), pp. 6–9. DOI: 10.1109/MIC.2013.42.

[8] Carl Boettiger. "An introduction to Docker for reproducible research". In: *SIGOPS Oper. Syst. Rev.* 49.1 (Jan. 2015). https://doi.org/10.1145/2723872.2723882, pp. 71–79. ISSN: 0163-5980. DOI: 10.1145/2723872.2723882.

[9] Lars Rasmusson and Diarmuid Corcoran. "Performance overhead of KVM on Linux 3.9 on ARM cortex-a15". In: *SIGBED Rev.* 11.2 (Sept. 2014). https://doi.org/10.1145/2668138.2668143, pp. 32–38. DOI: 10.1145/2668138.2668143.

[10] Ankita Desai et al. "Hypervisor: A survey on concepts and taxonomy". In: *International Journal of Innovative Technology and Exploring Engineering* 2.3 (2013), pp. 222–225.

[11] Stephen J. Bigelow. *What's the difference between type 1 vs. type 2 hypervisor?: TechTarget*. Jan. 2021. URL: https://www.techtarget.com/searchitoperations/tip/Whats-the-difference-between-Type-1-vs-Type-2-hypervisor.

[12] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[13] Jordan Shropshire. "Analysis of monolithic and microkernel architectures: Towards secure hypervisor design". In: *2014 47th Hawaii International Conference on System Sciences*. IEEE. 2014, pp. 5008–5017.

[14] Hasan Fayyad-Kazan, Luc Perneel, and Martin Timmerman. "Benchmarking the performance of microsoft hyper-v server, vmware esxi and xen hypervisors". In: *Journal of Emerging Trends in Computing and Information Sciences* 4.12 (2013), pp. 922–933.

[15] Pankaj Kadam and Nilesh Alone. "Review on KVM Hypervisor". In: *International Journal of Recent Technology and Engineering (IJRTE) ISSN* (2014), pp. 2277–3878.

[16] Xiaolong Wen et al. "Comparison of open-source cloud management platforms: OpenStack and OpenNebula". In: *2012 9th International Conference on Fuzzy Systems and Knowledge Discovery*. IEEE. 2012, pp. 2457–2461.

[17] David Freet et al. "Open source cloud management platforms and hypervisor technologies: A review and comparison". In: *SoutheastCon 2016* (2016), pp. 1–8.

[18] Tiago Rosado and Jorge Bernardino. "An overview of openstack architecture". In: *Proceedings of the 18th International Database Engineering & Applications Symposium*. 2014, pp. 366–367.

[19] OpenStack Foundation. *OpenStack Live Migration*. Feb. 2024. URL: https://docs.openstack.org/nova/pike/admin/live-migration-usage.html.

[20] Rakesh Kumar et al. "Apache cloudstack: Open source infrastructure as a service cloud computing platform". In: *Proceedings of the International Journal of advancement in Engineering technology, Management and Applied Science* 111 (2014), p. 116.

[21] Apache CloudStack. *CloudStack Live Migration*. Feb. 2024. URL: https://svn.apache.org/repos/asf/cloudstack/docsite/html/docs/en-US/Apache_CloudStack/4.2.0/html/Admin_Guide/manual-live-migration.html.

[22] Daniel Moritz et al. "Enhancing a virtual security lab with a private cloud framework". In: *Proceedings of 2013 IEEE International Conference on Teaching, Assessment and Learning for Engineering (TALE)*. IEEE. 2013, pp. 314–320.

[23] Younes Khair, Abdeslam Dennai, and Youssef Elmir. "An experimental performance evaluation of OpenNebula and eucalyptus cloud platform solutions". In: *Artificial Intelligence and Heuristics for Smart Energy Efficiency in Smart Cities: Case Study: Tipasa, Algeria*. Springer. 2022, pp. 450–457.

[24] Anurag Choudhary, Pradeep Kumar Verma, and Piyush Rai. "A walkthrough of Amazon Elastic Compute Cloud (Amazon EC2): A Review". In: *International Journal for Research in Applied Science and Engineering Technology* 9.11 (2021), pp. 93–97.

[25] Amazon Web Services. *Amazon EC2*. Feb. 2024. URL: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html.

[26] Pallavi Wankhede, Minaiy Talati, and Rutuja Chinchamalatpure. "Comparative study of cloud platforms-microsoft azure, google cloud platform and amazon EC2". In: *J. Res. Eng. Appl. Sci* 5.02 (2020), pp. 60–64.

[27] Marshall Copeland et al. "Getting started with Azure virtual machines". In: *Microsoft Azure: Planning, Deploying, and Managing Your Data center in the Cloud* (2015), pp. 147–176.

[28] Rawan Aljamal, Fahed Jubair, and Ali El-Mousa. "Analyzing the Cost-Effectiveness of Running HPC Applications on Microsoft Azure Virtual Machines". In: *2021 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology (JEEIT)*. IEEE. 2021, pp. 78–82.

[29] Microsoft. *Azure Virtual Machines*. Feb. 2024. URL: https://learn.microsoft.com/en-us/azure/virtual-machines/overview.

[30] TechTarget. *Google Compute Engine*. Feb. 2024. URL: https://www.techtarget.com/searchaws/definition/Google-Compute-Engine#:~:text=Google%20Compute%20Engine%20(GCE)%20is,compute%20clusters%20for%20that%20purpose..

[31] SPT Krishnan et al. "Google compute engine". In: *Building Your Next Big Thing with Google Cloud Platform: A Guide for Developers and Enterprise Architects* (2015), pp. 53–81.

[32] Solanke Vikas et al. "Private vs public cloud". In: *International Journal of Computer Science & Communication Networks* 3.2 (2013), p. 79.

[33] Guruh Aryotejo, Daniel Y Kristiyanto, et al. "Hybrid cloud: bridging of private and public cloud computing". In: *Journal of Physics: Conference Series*. Vol. 1025. 1. IOP Publishing. 2018, p. 012091.

[34] Khouloud Boukadi et al. "From VM to container: A linear program for outsourcing a business process to cloud containers". In: *On the Move to Meaningful Internet Systems. OTM 2017 Conferences: Confederated International Conferences: CoopIS, C&TC, and ODBASE 2017, Rhodes, Greece, October 23-27, 2017, Proceedings, Part I*. Springer. 2017, pp. 488–504.

[35] Nane Kratzke. "About microservices, containers and their underestimated impact on network performance". In: *arXiv preprint arXiv:1710.04049* (2017).

[36] Arnaldo Pereira Ferreira and Richard Sinnott. "A performance evaluation of containers running on managed kubernetes services". In: *2019 IEEE international conference on cloud computing technology and science (CloudCom)*. IEEE. 2019, pp. 199–208.

[37] Vitor Goncalves da Silva, Marite Kirikova, and Gundars Alksnis. "Containers for Virtualization: An Overview." In: *Appl. Comput. Syst.* 23.1 (2018), pp. 21–27.

[38] Docker. *Layers*. Feb. 2024. URL: https://docs.docker.com/build/guide/layers/.

[39] Docker. *Dockerfile reference*. Feb. 2024. URL: https://docs.docker.com/reference/dockerfile/.

[40] Carl Boettiger. “An introduction to Docker for reproducible research”. In: *ACM SIGOPS Operating Systems Review* 49.1 (2015), pp. 71–79.

[41] Steve Buchanan et al. “Container registries”. In: *Introducing Azure Kubernetes Service: A Practical Guide to Container Orchestration* (2020), pp. 17–34.

[42] Red Hat. *What is a container registry?* Feb. 2024. URL: https://www.redhat.com/en/topics/cloud-native-apps/what-is-a-container-registry.

[43] Open Container Initiative. *Open Container Initiative*. Feb. 2024. URL: https://opencontainers.org/.

[44] Lennart Espe et al. “Performance Evaluation of Container Runtimes.” In: *CLOSER*. 2020, pp. 273–281.

[45] Arun Gurta. “Kubernetes”. In: *Kubernetes. Retrieved May* 24 (2019), p. 2019.

[46] Kubernetes. *Custom Resource Definitions*. Feb. 2024. URL: https://kubernetes.io/docs/concepts/extend-kubernetes/api-extension/custom-resources/.

[47] Kakoli Banerjee et al. “A Survey on Kubernetes Policy Report Custom Resource Definition Kube-Bench Adapter”. In: *Advances in Data and Information Sciences: Proceedings of ICDIS 2022*. Springer, 2022, pp. 315–322.

[48] Prateek Sharma et al. “Containers and virtual machines at scale: A comparative study”. In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[49] Alessandro Randazzo and Ilenia Tinnirello. “Kata containers: An emerging architecture for enabling mec services in fast and secure way”. In: *2019 Sixth International Conference on Internet of Things: Systems, Management and Security (IOTSMS)*. IEEE. 2019, pp. 209–214.

[50] Shashank Mohan Jain and Shashank Mohan Jain. “Cgroups”. In: *Linux Containers and Virtualization: A Kernel Perspective* (2020), pp. 45–80.

[51] Amazon Web Services. *Noisy Neighbor*. Feb. 2024. URL: https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/noisy-neighbor.html.

[52] Prateek Sharma et al. “Containers and virtual machines at scale: A comparative study”. In: *Proceedings of the 17th international middleware conference*. 2016, pp. 1–13.

[53] Jinchun Kim et al. “Dynamic memory pressure aware ballooning”. In: *Proceedings of the 2015 International Symposium on Memory Systems*. 2015, pp. 103–112.

[54] Ishan Banerjee et al. “Memory overcommitment in the ESX server”. In: *VMware Technical Journal* 2.1 (2013), pp. 2–12.

[55] Christopher Clark et al. “Live migration of virtual machines”. In: *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*. 2005, pp. 273–286.

[56] Andrey Mirkin, Alexey Kuznetsov, and Kir Kolyshkin. “Containers checkpointing and live migration”. In: *Proceedings of the Linux Symposium*. Vol. 2. 2008, pp. 85–90.

[57] Radostin Stoyanov and Martin J Kollingbaum. “Efficient live migration of linux containers”. In: *High Performance Computing: ISC High Performance 2018 International Workshops, Frankfurt/Main, Germany, June 28, 2018, Revised Selected Papers 33*. Springer. 2018, pp. 184–193.

[58] CRIU. *CRIU*. Feb. 2024. URL: https://criu.org/.

[59] Red Hat. *Containers vs VMs*. Feb. 2024. URL: https://www.redhat.com/en/topics/containers/containers-vs-vms.

[60] Kong. *What Are Virtual Machines (VMs)?* Feb. 2024. URL: https://konghq.com/learning-center/service-mesh/virtual-machines.

[61] Chris Greamo and Anup Ghosh. “Sandboxing and virtualization: Modern tools for combating malware”. In: *IEEE Security & Privacy* 9.2 (2011), pp. 79–82.

[62] Tripwire. *What is Configuration Drift?* Feb. 2024. URL: https://www.tripwire.com/state-of-security/what-is-configuration-drift.

[63] Shared Rotlevi. *Configuration Drift Explained*. Feb. 2024. URL: https://www.wiz.io/academy/configuration-drift.

[64] Sysdig. *Containers vs. Virtual Machines – Making an Informed Decision*. Feb. 2024. URL: https://sysdig.com/learn-cloud-native/kubernetes-security/containers-vs-virtual-machines-making-an-informed-decision/.

[65] Dennis Gannon, Roger Barga, and Neel Sundaresan. “Cloud-native applications”. In: *IEEE Cloud Computing* 4.5 (2017), pp. 16–21.

[66] Microsoft. *Virtual machines: virtual computers within computers*. Feb. 2024. URL: https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-a-virtual-machine.

[67] Ramzi Debab and Walid Khaled Hidouci. "Containers runtimes war: a comparative study". In: *Proceedings of the Future Technologies Conference (FTC) 2020, Volume 2*. Springer. 2021, pp. 135–161.

[68] Anjali, Tyler Caraza-Harter, and Michael M Swift. "Blending containers and virtual machines: a study of firecracker and gVisor". In: *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 2020, pp. 101–113.

[69] Anjali, Tyler Caraza-Harter, and Michael M Swift. "Blending containers and virtual machines: a study of firecracker and gVisor". In: *Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. 2020, pp. 101–113.

[70] Kubernetes. *kube-scheduler*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/reference/command-line-tools-reference/kube-scheduler/.

[71] KubeVirt. *KubeVirt Architecture*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/architecture/.

[72] KubeVirt. *API Validation*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/api_validation/.

[73] KubeVirt. *Snapshot Restore API*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/snapshot_restore_api/.

[74] KubeVirt. *Live Migration*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/live_migration/.

[75] Kubernetes. *Persistent Volumes*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/concepts/storage/persistent-volumes/.

[76] Kubernetes. *Controllers*. Accessed 2024-02-29. Feb. 2024. URL: https://kubernetes.io/docs/concepts/architecture/controller/.

[77] KubeVirt. *virtctl*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/operations/virtctl_client_tool/.

[78] Florian Beetz and Simon Harrer. "Gitops: The evolution of devops?" In: *IEEE Software* 39.4 (2021), pp. 70–75.

[79] Platform9. *KubeVirt: How to Run VMs on Kubernetes*. Tech. rep. Apr. 2020. URL: https://platform9.com/wp-content/uploads/2020/05/KubeVirt-How-to-Run-VMs-on-Kubernetes.pdf.

[80] KubeVirt. *Comparing KubeVirt to other technologies*. Accessed 2024-02-29. 18 2017. URL: https://kubevirt.io/2017/technology-comparison.html.

[81] KubeVirt. *Network Binding Plugins*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/network_binding_plugins/.

[82] KubeVirt. *Disks and Volumes*. Accessed 2024-02-29. Feb. 2024. URL: https://kubevirt.io/user-guide/virtual_machines/disks_and_volumes/.

Appendix A

Something Extra